Group 4 - CloudNine Systems

Milestone 1: Draft Analysis Report

Status: proposed 30-table design for team review. Revised October 10, 2026. The roster and actual work assignments remain pending.

1. Scenario and purpose

CloudNine Systems is a fictional provider of shared cloud infrastructure. Individuals, teams and organizations reserve resources, deploy workloads, consume compute and ancillary services, and receive bills. Staff must coordinate changing capacity, failures, maintenance, client priorities and financial responsibility.

Our proposed database connects four stages: requested capacity, actual resource assignment, measured consumption and billing. It should explain who used which resources at a given time, whether future reservations can be honored, how deployments changed and how charges were distributed. Example data will be synthetic.

2. Scope and schema overview

The design contains 30 tables across six modules. The ERD PDF, Mermaid source, schema JSON and data dictionary describe the same model. Earlier 42-table and 50-table proposals are superseded. We retain the operational and billing backbone while explaining advanced features as scope limitations rather than creating a separate subsystem for each feature.

Clients and people (4 tables): CLIENT represents individuals, teams and organizations; an optional parent links departments or teams to an organization. USER_ACCOUNT records people and distinguishes client, staff and dual-role accounts. CLIENT_MEMBERSHIP records membership and roles over time. PROJECT groups work for a client. A payer can be a different client from the resource user.

Infrastructure and history (8 tables): REGION, AVAILABILITY_ZONE and DATA_CENTER describe location. RESOURCE_TYPE describes intended usage. RESOURCE identifies a managed physical asset or logical cluster. RESOURCE_CONFIGURATION preserves tier and scalar capacities over time. RESOURCE_STATE records operational, maintenance, repair and failure intervals. RESOURCE_RELATIONSHIP represents components and dependencies, including shared storage or networking.

Reservations and revisions (4 tables): RESERVATION identifies the requesting client. RESERVATION_REVISION preserves complete successive versions of requested dates. RESERVATION_ITEM specifies resource type, count, minimum capacities and location requirements. RESERVATION_SERVICE links requested ancillary services to an item. A reservation need not have a physical assignment yet.

Deployments and assignments (4 tables): DEPLOYMENT connects actual activity to a reservation and optionally a project. DEPLOYMENT_EVENT records scaling, priority, pause, resume and migration decisions. RESOURCE_ALLOCATION assigns one configuration over time with a reserved capacity fraction and links replacement assignments. DEPLOYMENT_SERVICE identifies each provisioned compute or ancillary offering.

Services, rates and metering (3 tables): SERVICE defines an offering and one billing unit. Distinct priced offerings, such as general compute and GPU compute, have distinct service records. SERVICE_RATE stores effective-dated standard prices. USAGE_RECORD records estimated or actual consumption for a deployment service, optionally identifying a resource allocation and a correction predecessor.

Billing and invoices (7 tables): BILLING_ACCOUNT identifies the payer and contact. BILLING_RULE applies an effective sharing agreement to a deployment, and BILLING_RULE_SHARE assigns account fractions. CHARGE records a usage charge, surcharge or credit. CHARGE_SHARE retains the actual monetary allocation. INVOICE groups final charges for an account; INVOICE_LINE links charge shares to invoices without duplicating their payer.

3. Main design decisions

Separate reservation from allocation. A client can request two resources next month before staff choose specific assets. Reservation items retain requested capacities, while allocations identify configurations and actual intervals. One reservation can support several deployments, and a deployment can span zones through multiple allocations.

Preserve time-based history. A request change creates a new reservation revision. Reconfiguration creates a new configuration interval. Migration closes an old allocation and creates a replacement. Availability is derived from operational state and allocations, rather than one status field combining maintenance, occupancy and future reservation.

Use fixed, atomic capacity fields. Configuration and request rows use vCPU count, memory GiB, storage GiB and network Gbps. These are separate scalar properties, not repeated groups or a serialized capacity description. Zero means the dimension is not provided or requested. A proportional capacity fraction reserves the same share of each dimension of a configuration. Independent multidimensional partitioning is outside the reduced model.

Separate consumption from service pricing. Compute, storage, traffic, backups, monitoring and support are service offerings. Each service has one canonical billing unit, such as vCPU-hour, GiB-hour, GiB transferred or support-hour. Metering does not repeat that unit. Standard rates are effective-dated; service variants can distinguish different resource classes or service levels.

Keep financial records explainable. A metered charge references its usage and applied rate. A direct surcharge references its deployment service. A credit or adjustment references its original charge. Exactly one of these three contexts is populated, so deployment ownership is derived rather than copied. Account shares are frozen when a charge is finalized; later sharing changes do not rewrite history.

4. Assumptions and integrity rules

A1. Each data center belongs to one zone and each zone to one region. A resource has one administrative home; a logical cluster may have components in other zones. Resource identifiers are unique within their data center. CLIENT.parent_client_id forms a single-parent hierarchy; cycles are prohibited. Membership roles are separate from staff roles.

A2. All money is USD and all timestamps are UTC. Intervals are half-open [start, end), with null end meaning open-ended. End must exceed start. Decimal amounts use fixed precision. Quantities and capacity values are nonnegative, and reservation item counts are positive integers.

A3. Each published reservation revision contains at least one item. Revision numbers are unique within a reservation. The highest approved revision is current; cancellation is represented by reservation status. Approvers and staff-only actions must reference active staff or dual-role user accounts. Changing a request requires reviewing affected future allocations.

A4. A reservation item may choose at most one location level: region, zone or data center. Broader locations are derived. location_mode distinguishes mandatory requirements from preferences. Allocations violating mandatory location or capacity requirements require a newly approved revision; preferred substitutions retain a reason.

A5. Every allocation identifies one configuration; resource_id is derived from that configuration. The configuration interval must cover the assignment. An allocation's item must belong to the deployment's reservation. Its fractional capacity must meet the item's minimum capacities. Configuration and operational-state intervals cannot overlap for the same resource.

A6. capacity_fraction is greater than zero and at most one. The sum across overlapping active or planned allocations cannot exceed one per physical resource; an exclusive assignment uses one. Maintenance or failure blocks usable capacity. Active logical cluster relationships are expanded when checking allocations, so assigning a cluster and its dedicated components cannot consume capacity twice. Only one accounting level is metered for the same consumption.

A7. A usage row belongs to one deployment service; allocation_id is optional for non-resource services. If supplied, the allocation must belong to the same deployment, and the metered interval must fall inside both provisioning and assignment intervals. Compute usage requires an allocation. A service's billing unit is immutable once referenced; a unit change creates a new offering.

A8. record_type is estimated or actual. A correction creates a new row referencing the previous usage row; chains cannot cycle or branch and must retain source, interval and record type. Only the latest actual row contributes to finalized billing. Estimates remain separate and never add to actual consumption totals.

A9. Standard service-rate intervals do not overlap for the same service. Each usage row used for pricing falls within one rate interval; consumption crossing a rate boundary is split before charging. The applied rate must match the usage service. Standard amount equals usage quantity times unit price, rounded to two decimal places. A metered usage row is linked to at most one charge; a provisional estimate charge is voided when a separate actual record is finalized.

A10. CHARGE populates exactly one of usage_id, deployment_service_id and adjustment_of_charge_id. Metered charges require service_rate_id; direct surcharges and adjustments leave it null. Credits and surcharges require an internal staff authorizer and reason. A credit has a negative amount. Adjustment chains cannot cycle. Corrections to already finalized usage are handled with an authorized reversing adjustment and a replacement charge, retaining the original posted facts.

A11. Each deployment has at most one effective billing rule at a time. Every rule has positive shares totaling one. Every final charge has account shares totaling its amount. Cross-client payment is allowed by agreement. The applicable rule is selected using the usage period start, the surcharge creation time or the original charge's allocation for reversals. Later rule changes cannot modify posted shares. Monetary rounding residuals go to the largest share using account ID to break ties.

A12. Each charge share appears on at most one invoice, and its payer must equal the invoice account. Only final charges may be invoiced. Invoice totals are computed from lines. Payments, cash refunds and invoice settlement balances are outside this version; invoice status describes document issuance, voiding or crediting, not proof of payment.

A13. Additional unique keys are USER_ACCOUNT.email; REGION.name; RESOURCE_TYPE.name; SERVICE.name; AVAILABILITY_ZONE(region_id, name); DATA_CENTER(zone_id, name); RESOURCE(data_center_id, local_identifier); RESERVATION_REVISION(reservation_id, revision_number); SERVICE_RATE(service_id, valid_from); BILLING_RULE(deployment_id, valid_from); CHARGE.usage_id when populated; CHARGE_SHARE(charge_id, billing_account_id); and USAGE_RECORD.supersedes_usage_id when populated. Composite PKs in the ERD enforce reservation-service, billing-rule-account and invoice-line association uniqueness. Membership and dependency intervals for the same pair and role/type cannot overlap.

A14. The course interface is read-only. Account type, memberships and deployment confidentiality provide metadata for an application policy: clients may view authorized memberships, designated internal staff may view operational data, and sensitive details are filtered. This reduced model does not implement credentials or arbitrary object-level grants. Synthetic data contains no real credentials or personal records.

PK, FK, UNIQUE and row CHECK constraints enforce local rules. Interval overlap, capacity checks, mandatory child counts, sharing totals and cross-table ownership require transactional application checks or database triggers. The ERD specifies these rules but does not implement them.

5. Third Normal Form rationale

We evaluate the declared business dependencies, not just the presence of surrogate IDs. Under this scope, each primary or alternate key determines its relation's non-key attributes. Role labels, tier labels and explanatory notes do not imply additional lookup attributes stored in the same row.

1NF: each attribute holds one scalar value. Members, requested services, allocations, dependencies and billing recipients occupy separate rows. Capacity columns describe distinct dimensions; they do not encode a list. Agreement and workload notes are single descriptive texts and are not used to store machine-readable collections of contracts or workload entities.

2NF: attributes of composite-key relations depend on the complete key. In RESERVATION_SERVICE, (reservation_item_id, service_id) determines requested quantity. In BILLING_RULE_SHARE, (billing_rule_id, billing_account_id) determines share_fraction. INVOICE_LINE has one charge_share_id key and identifies its invoice. No association row copies a name or description determined by only one key component.

3NF: resource geography is stored through RESOURCE -> DATA_CENTER -> AVAILABILITY_ZONE -> REGION, without repeating ancestor IDs in RESOURCE. Allocation references configuration_id without repeating resource_id. Reservation item location columns are mutually exclusive. Usage references its deployment service without copying deployment_id, service_id or billing_unit. SERVICE_RATE stores the price history separately from SERVICE.

A charge uses one context: usage, direct service or original charge. It does not copy the deployment derivable from that context. Its optional usage_id is unique, and is an alternate identifier for a metered charge. Rate selection and amount for such a row are therefore dependent on a candidate identifier, not a repeated non-key determinant. Manual charge contexts omit service_rate_id. Authorized adjustments are separately posted accounting facts.

CHARGE_SHARE omits invoice_id; INVOICE_LINE stores invoice membership, avoiding a stored invoice-to-account dependency in the share row. BILLING_RULE has deployment scope only, so it does not repeat its project's or client's identifiers. Billing totals, current priority, resource occupancy and available capacity are derived views. Under these declared dependencies and XOR rules, the logical relations satisfy 3NF. New dependencies introduced during implementation require another normalization review.

6. Requirements coverage and deliberate simplifications

Retained directly: clients, teams, organizations, contacts, staff roles and project ownership; resource types, tiers, geographic hierarchy, configurations, dependencies and states; advance requests, revisions, services, allocation history and substitutions; deployment changes, priorities, estimated and actual consumption; distinct payers, cost sharing, standard rate history, posted charges, credits and invoices.

Contracts, SLA commitments, qualification and deposits: reservation agreement notes and client qualification notes explain manual decisions. Separate contracts, deposit amounts, payment history and eligibility calculations

are excluded. This version cannot compute funded deposits or automate SLA compliance; those decisions occur outside the course database.

Complex pricing and sharing: standard service prices are implemented. Time-of-day tiers, duration and volume pricing, long-term or bundled discounts are not evaluated automatically. Staff can document an agreed surcharge or credit. Sharing is configured per deployment; client-wide or project-wide defaults are entered as deployment rules, without a separate inheritance engine.

Workloads and service options: a deployment may describe several cooperating workloads in a narrative, but individual workload inventory and workload-specific costs are not queried. Distinct billable service options become separate SERVICE offerings. RESOURCE_SERVICE capability mapping is omitted; staff verify supported service offerings during allocation.

Audit and access: deployment changes, approval actors, resource-state recorders, allocation reasons and charge authorizers provide operational accountability. A complete authentication log, general audit subsystem and configurable permission grants are deferred. Current organizational parentage is stored; membership and operational history are retained, but historical organizational restructuring is not modeled.

Payments and refunds: credits can reduce posted charges, but cash refunds and settlement are not represented. Graphical topology can be derived from resource relationships; an interactive zoomable view is deferred. Allocation optimization, autoscaling decisions, fairness scoring and live metering collection remain external processes whose results are recorded.

These limitations account for the omitted scenario details explicitly. They reduce implementation effort while retaining an end-to-end reservation-to-invoice path. They are scope proposals for instructor feedback, not claims that every CloudNine feature has been fully implemented.

7. Validation example and subsequent work

Example: a research team reserves two general-purpose instances for a project. Staff approve the first revision and create two allocations meeting its minimum capacities. A compute service records hourly consumption. A failure closes one allocation; a replacement allocation and migration event preserve the reason. Actual usage generates charges, a 60/40 deployment rule splits them between two billing accounts, and each account receives its own invoice. A staff-authorized credit compensates for the disruption. Payers may differ from the requesting team.

Milestone 1 delivers design documents only. Subsequent work will select the course DBMS, implement constraints, and generate at least one quarter of synthetic activity. Proposed aggregate or multi-table queries cover utilization, future reservation demand, unavailable capacity, migration frequency, estimate-versus-actual usage, client spending, cost-center sharing and disruption credits. The planned interface uses one or two read-only pages.

The team statement retains provisional work packages until member names, availability, responsibilities and acceptance are confirmed. All members should review the reduced scope and be able to explain the validation example before submission.

Source basis

Course handouts: Group Project - DB.pdf (seven pages) and Group Project_ CloudNine Systems.pdf (five pages), plus the supplied Milestone 1 instructions. Table names, assumptions and simplifications are this team's proposed interpretation of the scenario.